

ARM

A PROJECT REPORT

Submitted by

JOEL JOJU [SCM23MCA-2020]

to

The APJ Abdul Kalam Technological University
in partial fulfillment of the requirements for the award of the Degree of
Master of Computer Application



Department of Computer Science and Engineering

SCMS SCHOOL OF ENGINEERING AND TECHNOLOGY

(Affiliated to APJ Abdul Kalam Technological University)

VIDYA NAGAR, PALISSERY, KARUKUTTY

ERNAKULAM - 683582

May 2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCMS SCHOOL OF ENGINEERING AND TECHNOLOGY

(Affiliated to APJ Abdul Kalam Technological University)

VIDYA NAGAR, PALISSERY, KARUKUTTY

ERNAKULAM - 683582



CERTIFICATE

This is to certify that the report entitled 'ARM' submitted by **JOEL JOJU** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of Master of Computer Application is a bonafide record of the project work carried out by him under my guidance and supervision.

Project Guide

Ms. Doney Daniel

Head Of The Department

Dr. Manish T I

Project Coordinator

Ms. Greshma P Sebastian

External Examiner

ACKNOWLEDGEMENT

I am greatly indebted to **Dr. Anitha G Pillai**, Principal, SSET, Karukutty, Ernakulam, who wholeheartedly granted me the permission to do project.

I would like to thank **Dr. Manish T I**, Head of the Department of Computer Science and Engineering, SSET, for his moral support and guidance throughout the project.

I would like to thank my guide, **Ms. Doney Daniel**, Assistant Professor, Department of Computer Science and Engineering, and project coordinator, **Ms. Greshma P Sebastian**, Department of Computer Science and Engineering, SSET who has given me valuable guidance and support all the way.

I would like to express my sincere gratitude to all the teachers of Computer Science Department who gave me moral and technical support. I would like to thank the supporting staff in the Computer lab whose dedicated work kept the lab working smoothly, thus enabling me to have access to various resources which helped me understand more about the project topic. I would also like to thank friends and family members for providing me with necessary resources and support. Last but not the least, I would like to thank God Almighty for helping me to do my project hassle free.

ABSTRACT

A robotic system called gesture-controlled robotic arm operates through hand movements to execute particular actions. The technology applies machine learning and computer vision to recognize hand movements before changing them into commands for the robotic arm. The system enables incident-free command management that benefits multiple domains from factory automation to robot-assisted tech and human-robot communication.

A robotic arm system has been developed through the combination of OpenCV and MediaPipe for real-time hand detection purposes. The system identifies hand landmarks which are used to set servo motor positions to enable robotic arm motion control. A precision servomotor system operates on an Arduino Uno CH340C combined with PCA9685 servo driver. The BlazePalm model from MediaPipe functions as the hand tracking system to detect 21 important hand landmarks for precise gesture recognition. A connection between the Python-based gesture recognition system and Arduino exists through the implementation of PySerial. The robotic arm shows precise control through this project which can lead to uses in healthcare together with manufacturing and virtual reality applications.

TABLE OF CONTENTS

CONTENTS	PAGE NO.
Acknowledgement	i
Abstract	ii
List Of Figures	iv
Chapter-1: Introduction	1
1.1 General Background	1
1.2 Objective	1
1.3 Scope	3
1.4 Organization Of Report	5
Chapter-2: Literature Survey	6
Chapter-3: System Requirements And Specifications	8
3.1 Hardware Requirements	8
3.2 Software Specifications	8
3.3 Software Tools & Platforms	9
Chapter-4: System Study And Analysis	10
4.1 Proposed System	10
Chapter-5: System Design	12
5.1 Introduction	12
5.2 System Architecture	12
5.3 Module Description	13
5.4 Diagrams	16
Chapter-6: Screenshots And Images	18
Chapter-7: Conclusion And Future Scopes	24
References	26

LIST OF FIGURES

NO.	TITLE	PAGE NO.
5.1	Arm Architecture	16
5.2	Circuit Diagram	17
6.1	Side View 1	18
6.2	Front View	18
6.3	Side View 2	19
6.4	Closed Palm	19
6.5	Open Palm	20
6.6	Left Servo Activated	20
6.7	Right Servo Activated	21
6.8	Arm Move To Front	21
6.9	Arm Move To Back	22
6.10	Arm Rotate Left	22
6.11	Arm Rotate Right	23

CHAPTER-1

INTRODUCTION

1.1 GENERAL BACKGROUND

The purpose of this work is to create a robotic arm system which uses computer-vision based gesture commands instead of physical controls. Robotics control systems that function traditionally depend on advanced interface elements supported by physical devices that limit continuous human-machine interoperation. Real-time basic hand motions can operate the robotic arm through users by linking laptop cameras to Arduino microcontrollers with servo motors which run under OpenCV and MediaPipe software framework management. Real-time hand movements detection by the system allows precise interpretation leading to the robotic arm control operation sequence generation. The system combines natural control methods with simpler usability procedures while eliminating the expense related to specialized control hardware implementation.

A gesture detection system built for vision operates through servo control technology that joins the PCA9685 driver with Arduino Uno CH340C. Through the communication layer of PySerial the computer can transmit real-time command commands to the robotic arm. The project functions for multiple industrial sectors and assistive technology domains while providing interactive systems an easy way to manage human-machine interface operations.

1.2 OBJECTIVE

Natural human-to-robot interactions will become possible through the development of arm project that uses gesture control to operate robotic arm. The project aims at achieving following main objectives:

- **Enhancing Intuitive Human-Robot Interaction:** Enable users to control a robotic arm using natural hand gestures, eliminating the need for conventional manual controllers and creating a seamless interaction interface.

- **Improving Precision and Responsiveness:** Achieve real-time gesture detection and mapping using OpenCV and MediaPipe, ensuring that the arm's movements—mediated by precise servo control via an Arduino and PCA9685 driver—are both smooth and accurate.
- **Streamlining Robotic Control Systems:** Integrate computer vision, machine learning, and efficient servo actuation into a cohesive system that simplifies the overall control architecture, reducing complexity and potential points of failure.
- **Increasing System Flexibility and Scalability:** Develop a scalable and adaptable framework that can be extended to incorporate additional gestures, more advanced robotic mechanisms, or integration with IoT platforms for remote operation.
- **Reducing Cost and Complexity:** Utilize cost-effective, widely available hardware (e.g., Arduino Uno CH340C, MG90S servos, PCA9685) and open-source software libraries, making advanced robotic control accessible and economically viable.
- **Fostering Innovation in Robotics:** Serve as a testbed for the exploration and further development of advanced gesture recognition algorithms and AI-powered control strategies, paving the way for enhanced applications in industrial automation, healthcare, education, and assistive technology.
- **Enabling Autonomous and Remote Operation:** Lay the groundwork for future enhancements that could allow the robotic arm to be controlled remotely or operate autonomously in environments where human intervention is challenging.
- **Enhancing Research and Development:** Provide valuable insights into real-time computer vision-based control systems, contributing to the academic and practical advancements in gesture recognition and robotic manipulation.

By fulfilling these objectives, the project aims to create a highly intuitive and efficient robotic system that not only improves the ease of control but also opens new avenues for research and applications in various fields.

1.3 SCOPE

Different factors related to gesture recognition system complexity and robotic arm freedom degrees as well as hardware availability and project financing determine the scope of the Gesture-Controlled Robotic Arm project. Broadly, the scope encompasses the following areas:

System Design and Planning

- Understand the control requirements and user interactions for a gesture-controlled robotic arm.
- Develop the overall system architecture integrating computer vision, machine learning, and robotic actuation.

Hardware Selection and Integration

- Select and integrate key components such as a laptop camera, Arduino Uno CH340C, PCA9685 servo driver, and MG90S servos.
- Ensure proper interfacing between the microcontroller, servo driver, and servos for smooth operation.

Computer Vision and Gesture Recognition

- Implement hand tracking using OpenCV and MediaPipe, focusing on detecting and processing 21 hand landmarks.
- Develop algorithms to classify gestures and translate these into control commands for the robotic arm.

Communication and Control Logic

- Establish real-time communication between the vision system (Python-based) and the microcontroller using PySerial.
- Develop control logic to map detected gestures to precise servo commands, ensuring responsive and accurate movement.

Software Development and Data Processing

- Create software modules for image processing, gesture classification, and servo control.
- Optimize algorithms for real-time performance and robustness against varying environmental conditions.

System Integration and Testing

- Integrate all hardware and software components into a robotic system.
- Conduct thorough testing in real-time scenarios to evaluate responsiveness, accuracy, and reliability of gesture recognition and robotic movement

User Interface and Feedback

- Develop a user interface or an .exe file for the robotic arm.
- Ensure that the system's behavior is intuitive and easy for users to interact with.

Scalability and Future Enhancements

- Design the system to be scalable, allowing for additional degrees of freedom or more complex gesture recognition in the future.
- Plan for potential integration of additional sensors or advanced AI models for enhanced control.

Safety and Reliability

- Incorporate mechanisms such as default positions and timeouts to ensure safe operation.
- Ensure that the robotic arm operates within safe mechanical limits and responds predictably to gesture inputs.

Documentation

- Prepare comprehensive documentation covering system design, hardware setup, software architecture, and operating procedures.

1.4 Organization of Report

The report divides content into seven parts which concentrate on essential project features. The project introduction through Chapter 1 reveals project origins and descriptions of its causes and aims with its boundaries defined. An extensive review of gesture identification approaches and robotic control systems and relevant technology appears in Chapter 2. The development of the gesture-controlled robotic arm uses the system requirements and specifications documented in Chapter 3 along with described hardware and software components. The analysis section of Chapter 4 compares standard control interfaces to gesture-based methods and explains the role of computer vision technology in gesture detection. The system design section of Chapter 5 demonstrates how different system modules communicate with each other while explaining the architecture structure. Implementation screenshots together with system images are shown in Chapter 6 to display the robotic arm movement through gesture recognition. The final section of this report presents the report's results along with explained obstacles and proposed future development opportunities. The References part of the report includes complete listings for all sources used as well as those referenced in the document.

CHAPTER-2

LITERATURE SURVEY

Paterson and Aldabbagh proposed a gesture-controlled robotic arm utilizing OpenCV for real-time hand tracking and control [1]. The system used a laptop camera to detect hand movements, which were then mapped to servo motor actions via serial communication. The authors focused on enhancing the system's responsiveness and accuracy using OpenCV's contour and convex hull methods. Their study demonstrated a high degree of precision and real-time control, making it suitable for interactive robotic applications.

Wameed and Alkamachi developed a computer vision-based hand gesture recognition system to control a robotic arm. They employed OpenCV for image processing and MediaPipe for detecting and tracking hand landmarks [2]. The study emphasized the importance of low latency and high accuracy in real-time gesture detection. The system achieved high responsiveness and smooth motion control, demonstrating its potential for industrial and assistive robotics.

Qi et al. conducted a comprehensive review of computer vision-based hand gesture recognition techniques for human-robot interaction [3]. They analyzed different models, including convolutional neural networks (CNNs) and recurrent neural networks (RNNs), for their effectiveness in recognizing complex hand gestures. The study highlighted the challenges of real-time processing and variability in hand shapes, proposing improvements in feature extraction and model training.

Peral et al. presented an efficient hand gesture recognition system for human-robot interaction using a combination of convolutional neural networks (CNNs) and optical flow analysis [4]. Their model demonstrated high accuracy and adaptability to different lighting conditions and hand shapes. The study focused on optimizing computational efficiency to enable real-time processing on low-power hardware.

Huu and Minh proposed an artificial neural network (ANN)-based gesture recognition algorithm for smart-home applications [5]. The system used a multi-layer perceptron (MLP) model to classify hand gestures captured through a camera. The study highlighted the importance of training data quality and model complexity in improving recognition accuracy.

Jo et al. explored the application of MediaPipe for hand gesture recognition in virtual and augmented reality (VR/AR) environments [6]. They demonstrated how MediaPipe's hand landmark model could be adapted for intuitive user interaction in virtual spaces. The study emphasized the importance of reducing latency and increasing gesture recognition stability for enhanced user experience.

Zhang et al. introduced a deep learning-based approach using 3D Convolutional Neural Networks (3D CNNs) and Convolutional LSTM (Long Short-Term Memory) for learning spatiotemporal features in gesture recognition [7]. The model demonstrated superior accuracy in recognizing dynamic gestures but required significant computational power. The study highlighted the importance of integrating temporal information for accurate gesture classification.

Gao et al. proposed a dynamic hand gesture recognition model based on 3D hand pose estimation using a deep learning framework [8]. The system achieved real-time performance and high accuracy by combining depth-based pose estimation with CNN-based classification. The study emphasized the role of 3D information in improving gesture recognition accuracy and robustness.

CHAPTER-3

SYSTEM REQUIREMENTS AND SPECIFICATIONS

The both hardware and software components configuration is suggested here. Operating requirements for this project arise from processing demands and real-time performance targets used to define hardware and software specifications for installation success. The system needs well-defined hardware and software requirements to operate smoothly because it combines computer vision with machine learning and robotic control functions. This document contains two distinct parts, they are - Hardware Requirements and Software Requirements.

3.1 HARDWARE REQUIREMENTS

- Arduino Uno CH340C – Microcontroller for processing servo commands.
- PCA9685 Servo Driver – To control multiple servo motors with high accuracy.
- Servo Motors (MG90S) – 4 units – For arm movement and claw control.
- Laptop Camera – For real-time hand gesture recognition using OpenCV.
- 5V Voltage Regulator – To stabilize the power supply to the servo driver and Arduino.
- 12V DC Adapter – For powering the servo motors and other components.
- USB Cable – For data transmission between the Arduino and the computer.
- Jumper Cables – For connecting hardware components.
- 3D-Printed Robotic Arm Parts – Base, joints, and claw components.

3.2 SOFTWARE SPECIFICATIONS

- Operating System: Windows 10 or above (64-bit).
- IDE: Arduino IDE
- Python Editor: VS Code

3.3 SOFTWARE TOOLS & PLATFORMS

PROGRAMMING LANGUAGES

- Python Programming
- Arduino Programming

LIBRARIES AND FRAMEWORKS

- OpenCV – For image processing and contour detection.
- MediaPipe – For hand tracking and landmark detection.
- PySerial – For serial communication with the Arduino.

CHAPTER-4

SYSTEM STUDY AND ANALYSIS

System study involves a comprehensive analysis of existing control systems and the identification of areas that require improvement or modernization. In the current scenario, many robotic systems still rely on traditional control mechanisms such as joysticks, buttons, or remote controllers, which can be unintuitive and complex for users with little technical experience. Our project specifically aims to address this gap by replacing these conventional systems with a gesture-based interface that offers a more natural and seamless interaction.

The ARM project focuses on the utilization of computer vision and machine learning to detect hand movements and translate them into specific actions for a robotic arm. This shift toward intuitive gesture control enhances user interaction, reduces dependency on physical control devices, and opens up new possibilities for applications in various domains such as assistive technology, remote operations, and education.

The analysis phase of the project includes a study of the conventional robotic control systems, identification of challenges such as limited flexibility and precision, and research into alternative input methods using real-time video processing. The accuracy of the system is based on environmental factors such as lighting and hand positioning.

4.1 PROPOSED SYSTEM

Gesture Recognition Module

- A laptop camera is used to continuously capture live video feed of the user's hand.
- OpenCV (Open Source Computer Vision Library) is used for pre-processing the video stream, including background filtering and frame extraction.
- MediaPipe, a powerful machine learning-based framework developed by Google, is used for real-time hand tracking. It detects 20 hand landmarks such as finger joints and palm positions.

- The positions of these landmarks are then analyzed to identify specific gestures like an open palm, closed fist, thumbs up, etc.
- These gestures serve as input commands, which are passed to the next module for processing.

Data Processing and Communication Module

- The recognized hand gesture is mapped to corresponding robotic arm actions using algorithm.
- For example, a closed fist may correspond to a specific motor angle for gripping an object, while an open hand could release it.
- These mappings are converted into servo motor angles (in degrees), which define the position of each joint in the robotic arm.
- PySerial, a Python library for serial communication, is used to establish a real-time data link between the computer and the microcontroller.
- The gesture data is transmitted as formatted command signals via a USB serial connection.

Robotic Arm Control Module

- The Arduino microcontroller receives gesture commands through the serial interface and decodes them.
- The servo driver module is used to generate precise PWM signals, which control the angle of multiple servo motors simultaneously.
- Each servo motor corresponds to a different part of the robotic arm.
- Based on the received PWM signals, the servos respond with smooth and accurate movement, replicating the intended gesture.

This structured modular design allows the system to process live gestures and perform corresponding mechanical actions with minimal latency. This system provides more interactive and user-friendly interface, enabling users to control complex robotic tasks using simple, natural hand movements.

CHAPTER-5

SYSTEM DESIGN

5.1 INTRODUCTION

The design of an effective gesture-controlled robotic arm heavily relies on a well-developed system. The chapter presents the entire system infrastructure along with hardware and software integration together with the design methodology which brings system specifications into operational functionality. Detailed creative engineering and extensive planning during the design phase made all system elements including computer vision modules to servo actuation function properly together. The outcome of this stage is a comprehensive design specification that guides the development, implementation, and evaluation of the system.

5.2 SYSTEM ARCHITECTURE

Sensing and Data Acquisition:

Laptop camera captures real-time video of the user's hand gestures. After video frames are then processed by computer vision modules to extract relevant features.

Processing and Decision Making:

OpenCV is used for image pre-processing and MediaPipe to detect 20 hand landmarks. These landmarks are then analyzed to classify gestures, which are then mapped to corresponding servo angles. PySerial is used to transmit these commands via a serial connection to the microcontroller.

Actuation and Control:

The microcontroller receives the serial commands and, in coordination with the servo driver, generates precise PWM signals to control the 4 servo motors. These servos actuate the robotic arm's joints (base rotation, arm lift, wrist movement, and claw control) in response to the detected gestures.

5.3 MODULE DESCRIPTION

Gesture Recognition Module:

The module makes use of OpenCV to record video frames while the pre-trained hand tracking model from MediaPipe identifies and retrieves 20 hand landmarks. The data collected from hand gestures gets processed to detect particular hand postures that function as input directions.

Data Processing and Command Mapping Module:

The extracted hand landmarks go through specialized algorithms which transform the data points into accurate servo angle instructions. The model converts diverse hand movement patterns into exact signals which control the robotic arm.

Step 1: Extract Key Landmarks

MediaPipe detects 20 landmarks per hand, each with (x, y, z) coordinates. Key landmarks for finger movements:

- Wrist: landmarks(0)
- Thumb: landmarks(1)–(4)
- Index Finger: landmarks(5)–(8)
- Middle Finger: landmarks(9)–(12)
- Ring Finger: landmarks(13)–(16)
- Little Finger: landmarks(17)–(20)

Step 2: Calculate Joint Angles

1. Find Finger Bends

- Calculate the angle between the wrist, MCP joint, and PIP joint for each finger using the dot product formula.

$$\theta = \cos^{-1} \left(\frac{(A - B) \cdot (C - B)}{|A - B||C - B|} \right)$$

Where:

A=Base joint(MCP)

B=Middle joint(PIP)

C=Tip joint(DIP)

2. Thumb Rotation

Compare the x position of landmarks (1) and (4) to detect thumb outward/inward movement.

3. Map Angles to Servo Values

Normalize the calculated angles to match the servo range 0° to 180°.

Step 3: Convert to Servo Commands

- Scale Angles to Servo Range:

Python code: `servo_angle = int((angle / 180) * 90) # Adjust 90° midpoint`

- Send Data to Arduino via Serial Communication:

Python code: `ser.write(f"{servo1},{servo2},{servo3},{servo4}\n".encode())`

- Arduino Reads Serial and Moves Servos:

Python code: `Servo1.write(servo1);`

`Servo2.write(servo2);`

Communication Module:

Real-time servo commands can be transmitted through the combination of Python's environment on the host computer and the PySerial module and the Arduino microcontroller. Having this module enables reliable real-time transmission between these systems.

Control Module:

The Arduino system together with the PCA9685 servo driver transmits processed commands to produce PWM signals that control the MG90S servo motors. The physical robotic arm movements are triggered by this module through gesture inputs.

5.4 DIAGRAMS

ARCHITECTURE

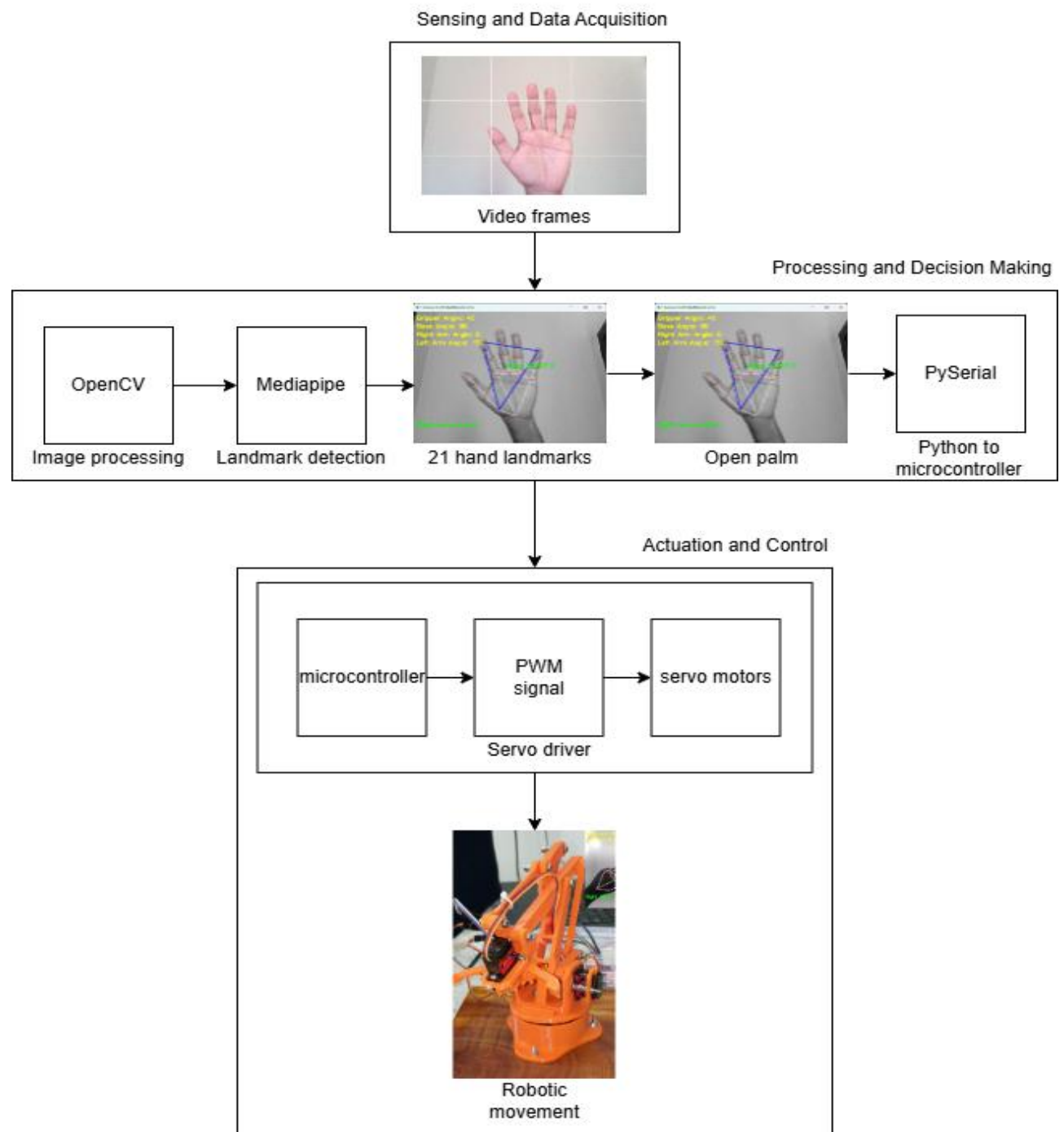


Fig 5.1 ARM Architecture

CIRCUIT DIAGRAM

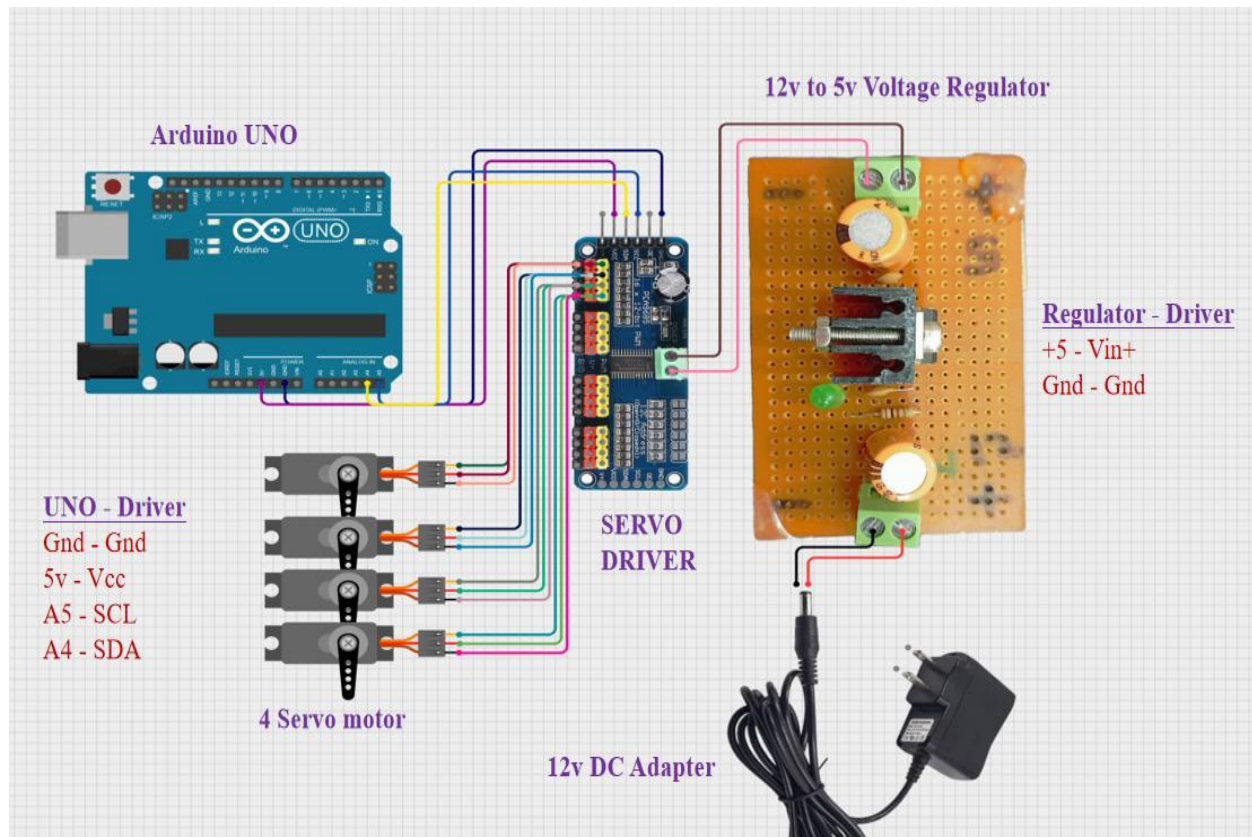


Fig 5.2 Circuit diagram

CHAPTER-6

SCREENSHOTS AND IMAGES

ARM Robot



Fig 6.1 Side view 1



Fig 6.2 Front view



Fig 6.3 Side view 2

SCREENSHOTS

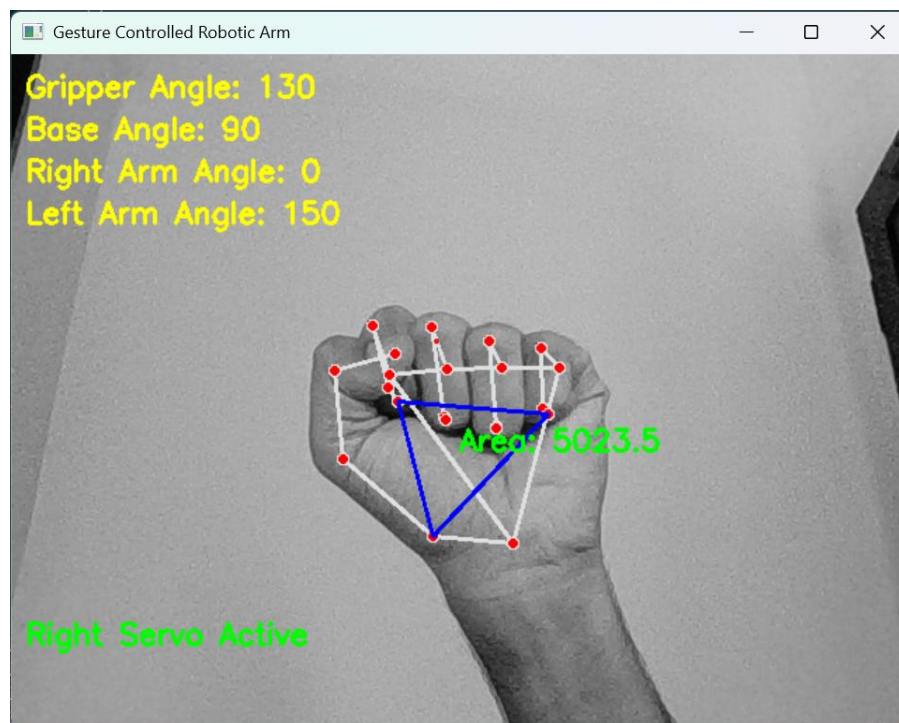


Fig 6.4 Closed palm

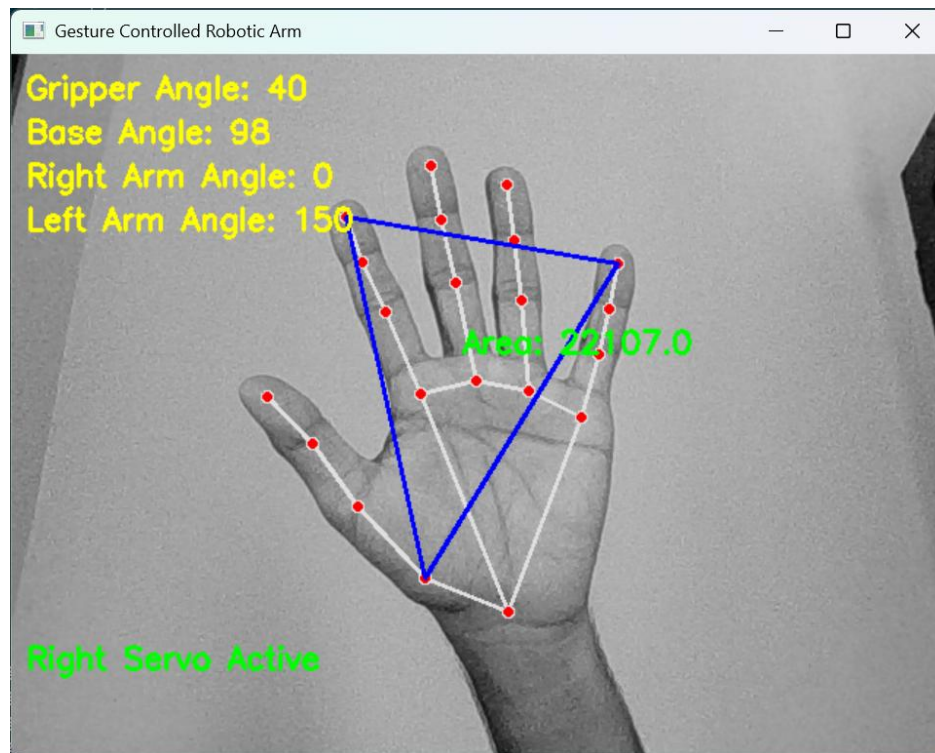


Fig 6.5 Open palm

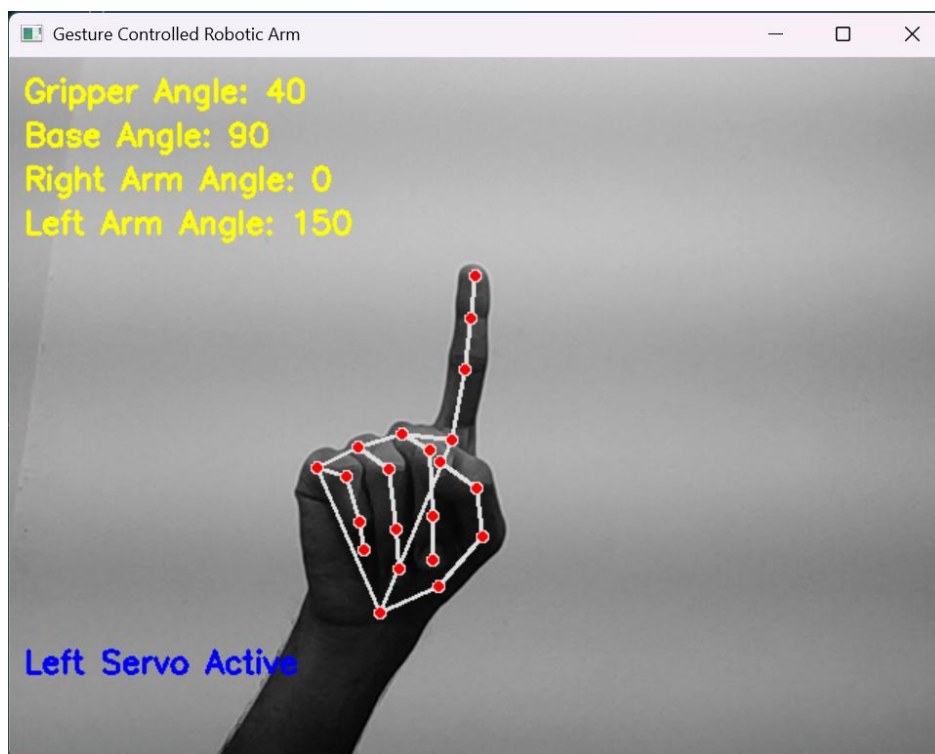


Fig 6.6 Left servo activated

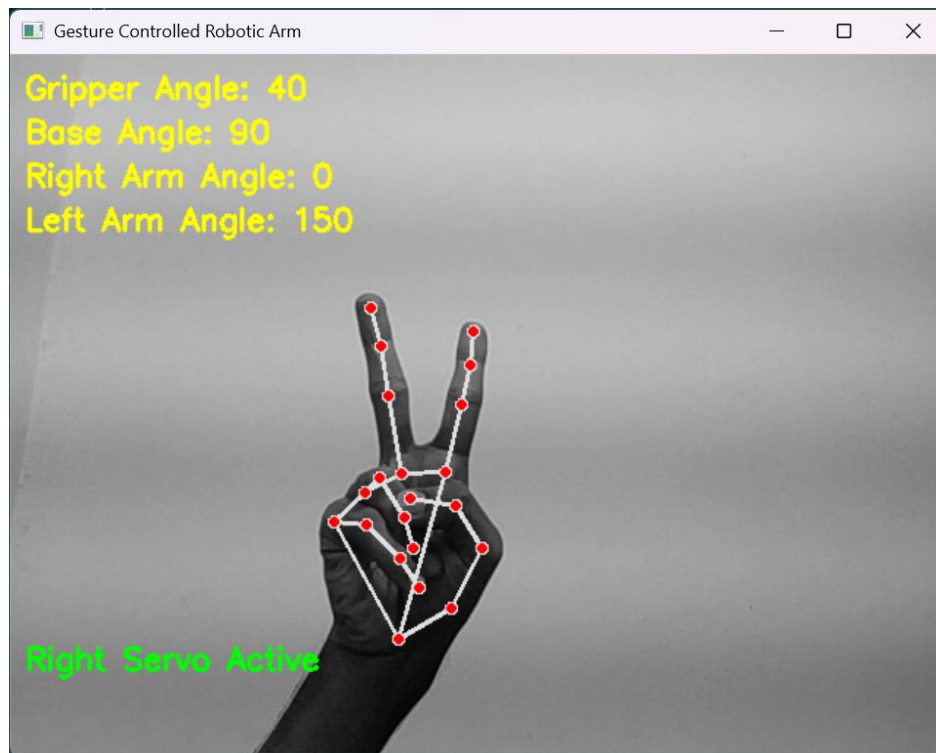


Fig 6.7 Right servo activated

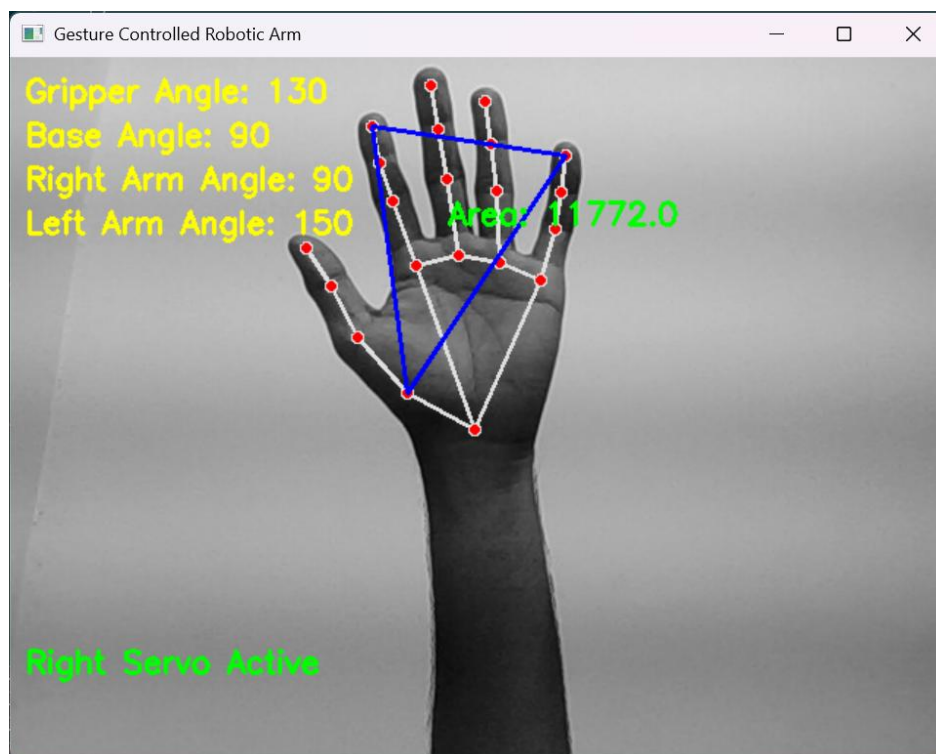


Fig 6.8 Arm move to front

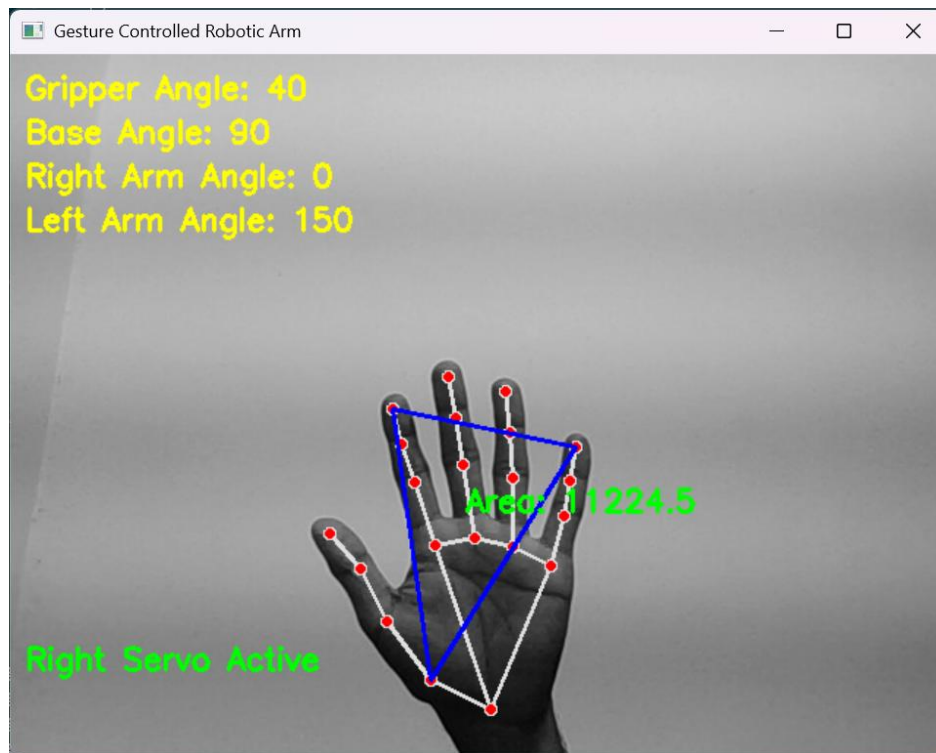


Fig 6.9 Arm move to back

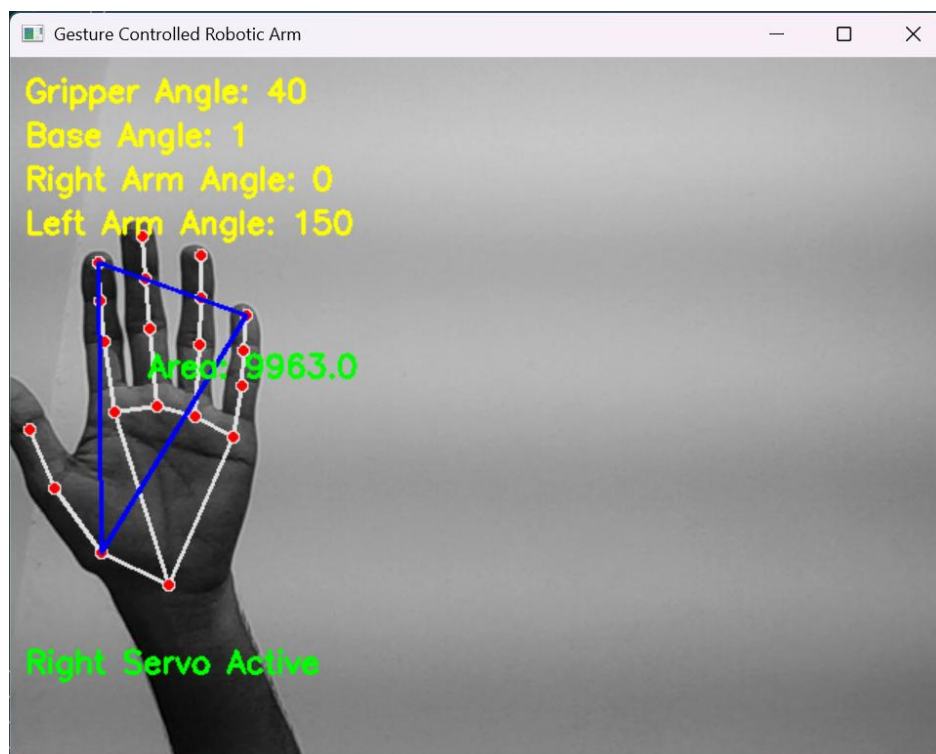


Fig 6.10 Arm rotate left

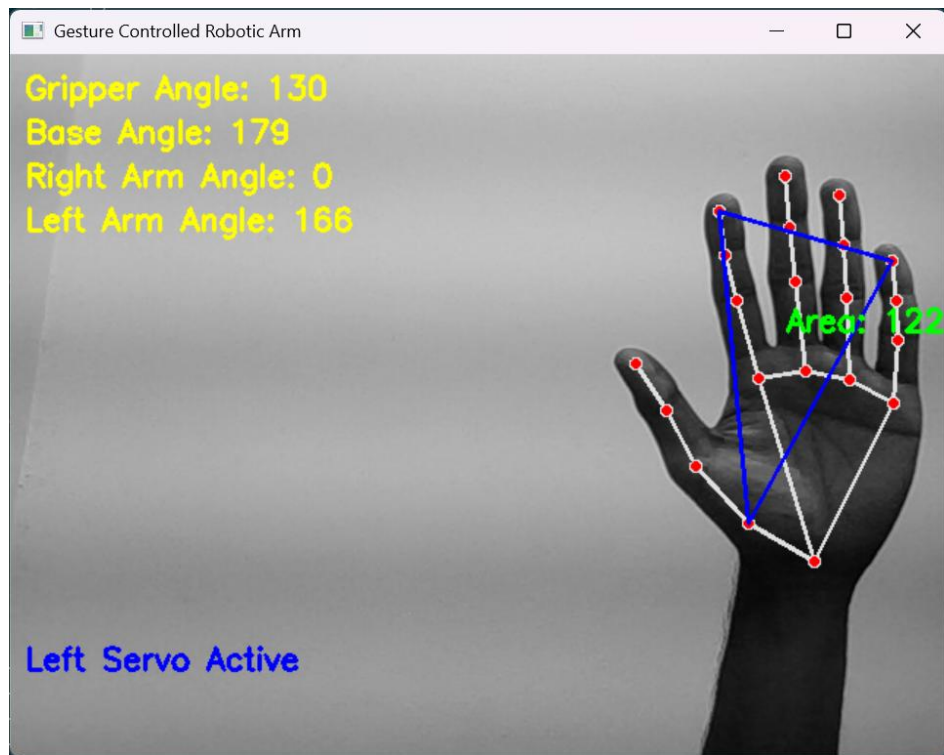


Fig 6.11 Arm rotate right

CHAPTER-7

CONCLUSION AND FUTURE SCOPES

7.1 CONCLUSION

A robotic arm has been successfully built in this project because it used real-time image capturing using laptop camera along with machine learning to detect hand gestures that converted into servo motor commands. OpenCV together with MediaPipe and PySerial form a system that detects hand landmarks using image frames and processes them to generate gesture classifications and transmits corresponding commands to control the robotic arm.

The completed implementation proves that human-computer interaction through gestures features strong potential to replace traditional controllers so operations become more intuitive and more natural. This approach has significant advantages, including:

- Contactless control:- useful for hazardous environments.
- Real-time response:- smooth operation.
- Scalability:- integration with more complex robotic applications.

CHALLENGES

- Gesture recognition accuracy depends on lighting conditions and background noise.
- Latency issues may arise due to computational processing time.
- Hardware limitations, such as servo motor precision and response time, can affect performance.

Other than these challenges, the project shows a practical and innovative application of computer vision and IoT in robotic control.

7.2 FUTURE SCOPE

- Using deep learning models to improve gesture classification accuracy.

- Ensuring better performance in various lighting and background conditions.
- Adding voice commands for alternative control.
- Using higher precision servo motors.
- Connecting to cloud server to allow remote monitoring and control from different locations.
- Expanding the system for medical applications, such as prosthetic arms.
- Enhancing the robotic arm for industrial automation and assembly line tasks.

By adding these advancements, the project can be developed into a more efficient, versatile, and intelligent robotic system, creating new possibilities in the fields such as automation, healthcare, and remote operations.

REFERENCES

- [1] Paterson, Jedidiah, and Ahmed Aldabbagh. "Gesture-controlled robotic arm utilizing opencv." 2021 3rd International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA). IEEE, 2021.
- [2] Wameed, M., and A. M. Alkamachi. "Hand gestures robotic control based on computer vision." *International Journal of Intelligent Systems and Applications in Engineering* 11.2 (2023): 1013-1021.
- [3] Qi, Jing, et al. "Computer vision-based hand gesture recognition for human-robot interaction: a review." *Complex & Intelligent Systems* 10.1 (2024): 1581-1606.
- [4] Peral, Marc, Alberto Sanfeliu, and Anaís Garrell. "Efficient hand gesture recognition for human-robot interaction." *IEEE Robotics and Automation Letters* 7.4 (2022): 10272-10279.
- [5] Huu, Phat Nguyen, and Quang Tran Minh. "An ANN-based gesture recognition algorithm for smart-home applications." *KSII Transactions on Internet and Information Systems (TIIS)* 14.5 (2020): 1967-1983.
- [6] Jo, Beom Jun, Seok-Kyoo Kim, and S. Kim. "Enhancing virtual and augmented reality interactions with a mediapipe-based hand gesture recognition user interface." *Ing. Syst. D'Inf* 28 (2023): 633-638.
- [7] Zhang, Liang, et al. "Learning spatiotemporal features using 3dcnn and convolutional lstm for gesture recognition." *Proceedings of the IEEE international conference on computer vision workshops*. 2017.
- [8] Gao, Qing, et al. "Dynamic hand gesture recognition based on 3D hand pose estimation for human-robot interaction." *IEEE Sensors Journal* 22.18 (2021): 17421-17430.